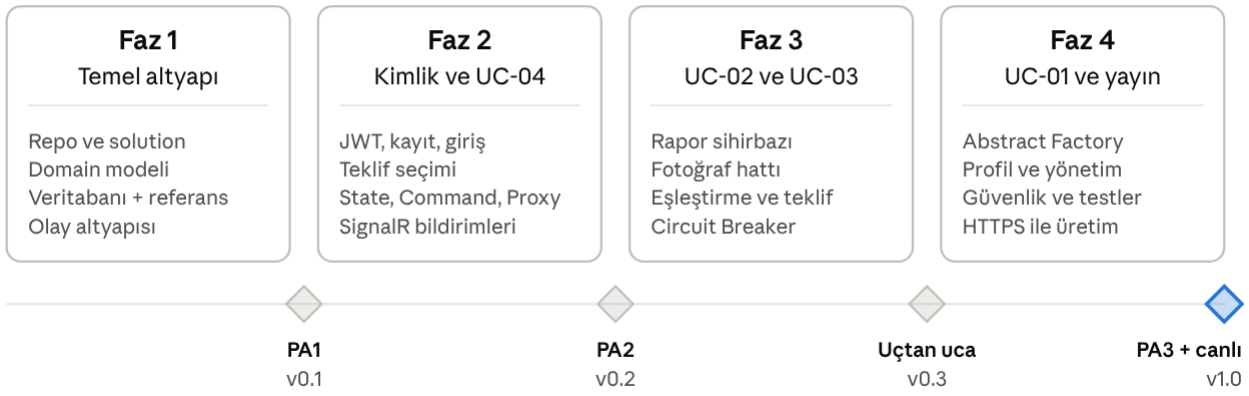
IndustryApp – 4 Fazlı Uygulama Rehberi

Sep 30, 2026 · @Bahadır İhsan Herdem

IndustryApp 4 fazda kurulur: Faz 1 temel altyapıyı, Faz 2 kimlik ve UC-04'ü (PA2), Faz 3 UC-02 ile UC-03'ü, Faz 4 UC-01'in tamamlanmasını, güvenliği, testleri ve üretime çıkışı (PA3) kapsar. Bu rehber, "IndustryApp – Teknik Tasarım ve Uygulama Rehberi" dokümanındaki her maddeyi uygulanma sırasına koyar; tablo kolonları, endpoint listesi ve interface imzaları için o dokümandaki bölüm numaralarına (T-6, T-11 gibi) atıf yapılır.

Bir bakışta

Her faz çalışan bir sürümle biter; PA3 ve canlı yayın Faz 4 sonunda



Yol haritası · 4 faz, 4 teslim kapısı

PA1 dokümanları Faz 1 ile paralel hazırlanır, PA2 Faz 2'nin çıktısıyla, PA3 ve canlı yayın Faz 4 sonunda teslim edilir; her faz bir sürüm etiketiyle kapanır.

Genel çalışma kuralları

Her adım aşağıdaki kurallara uyduğunda tamamlanmış sayılır. Kurallar bütün fazlarda aynıdır, bu yüzden faz bölümlerinde tekrar edilmez.

Bir adımın tamamlanma tanımı:

1. `dotnet build` uyarısız geçer (`TreatWarningsAsErrors`), `npm run build` hatasız geçer.
2. İlgili birim testleri yazılmış ve yeşil.
3. Desen uygulanan her sınıfın başında desen yorumu var (şablon aşağıda).
4. Sabit değer koda gömülmemiş; süre, sınır ve adresler `appsettings.json` üzerinden okunuyor (T-0.3, T-13.5).

5. Değişiklik UC numaralı commit ile gönderilmiş.

Commit ve branch biçimi:

- Branch: `uc-0n/<kısa-ad>` (örn. `uc-04/undo-selection`); altyapı işleri için `infra/<kısa-ad>`.
- Commit: `[UC-0n] <tür>: <açıklama>`; tür `feat`, `fix`, `test`, `docs`, `refactor` olabilir. Altyapıda `[INFRA]`.
- Her faz sonunda `main`'e birleştirilir ve `v0.<faz>` etiketi atılır (`v0.1` … `v0.4`).

Desen yorumu şablonu (PA2 zorunluluğu):

```
// [UC-04][PAT-04.2] Command
// Niyet   : Bu desenin burada çözdüğü problem.
// Neden   : Neden bu desen seçildi, alternatifi neydi.
// Kazanım : Esneklik, modülerlik veya bakım kolaylığına etkisi.
// Metrik  : Ölçülebilir etki ve onu doğrulayan test adı.
```

Biçimlendirme ve veri kuralları:

- Tarihler ekranda `dd/MM/yyyy` veya `dd/MM/yyyy HH:mm`; değer yoksa `'-'`.
- Enum veya kod değeri ekranda ham gösterilmez; Türkçe etikete çevrilir, karşılığı yoksa `'-'`.
- Tüm `null`, `undefined` ve boş metin durumları ele alınır; boş alan arayüzde `'-'` olarak görünür.
- REST ve SignalR JSON'u camelCase'e sabitlenir; frontend tipleri tek biçimle yazılır.
- Veritabanında seed yoktur. Referans veri `database/reference-data.sql` ile, işlem verisi ekranlardan oluşur (T-0.1).

Referanslar: Bu rehberde `T-n` ifadesi, teknik tasarım dokümanının n numaralı bölümünü gösterir. Örneğin tablo kolonları T-6'da, endpoint listesi T-11.3'te, servis imzaları T-8.3'tedir.

Faz 1 – Temel altyapı

Faz 1 sonunda repo, 6 projelik solution, tüm domain modeli, 21 tablolu veritabanı, referans veri, olay altyapısı ve Swagger'ı açılan boş bir API hazırdır. Hiçbir use case henüz çalışmaz; ama sonraki fazlarda yazılan her kod bu iskelete oturur.

Çıktı: `v0.1` etiketi · **Kapsadığı eski adımlar:** F0, F1, F2

1.1 Repo ve klasör yapısı

```
mkdir IndustryApp && cd IndustryApp && git init
mkdir -p database/dev deploy docs/PA1 docs/PA2 docs/PA3 docs/diagrams
src/backend src/frontend
dotnet new gitignore                      # .NET kuralları
curl -s
https://raw.githubusercontent.com/github/gitignore/main/Node.gitignore >>
.gitignore
dotnet new editorconfig
echo "database/dev/*.local.sql" >> .gitignore
```

`README.md` içine şimdilik 4 başlık yaz: Proje, Gereksinimler (.NET 8 SDK, Node 20, Docker), Yerel çalıştırma, Klasör yapısı. Klasör ağacı T-3'teki gibidir.

1.2 Solution ve projeler

```
cd src/backend
dotnet new sln -n IndustryApp
dotnet new classlib -n IndustryApp.Domain -f net8.0
dotnet new classlib -n IndustryApp.Application -f net8.0
dotnet new classlib -n IndustryApp.Infrastructure -f net8.0
dotnet new webapi   -n IndustryApp.WebApi -f net8.0 --use-controllers
dotnet new xunit    -n IndustryApp.UnitTests -o tests/IndustryApp.UnitTests -
f net8.0
dotnet new xunit    -n IndustryApp.IntegrationTests -o
tests/IndustryApp.IntegrationTests -f net8.0
dotnet sln add **/*.csproj

dotnet add IndustryApp.Application    reference IndustryApp.Domain
dotnet add IndustryApp.Infrastructure reference IndustryApp.Application
dotnet add IndustryApp.WebApi         reference IndustryApp.Application
IndustryApp.Infrastructure
dotnet add tests/IndustryApp.UnitTests        reference IndustryApp.Domain
IndustryApp.Application
dotnet add tests/IndustryApp.IntegrationTests reference IndustryApp.WebApi
```

`dotnet new` ile gelen `Class1.cs` ve `WeatherForecast` dosyalarını sil. `src/backend/Directory.Build.props` dosyasını oluştur; tüm projeler bu ayarları otomatik alır:

```
<Project>
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <LangVersion>latest</LangVersion>
  </PropertyGroup>
</Project>
```

1.3 NuGet paketleri

```
# Application
dotnet add IndustryApp.Application package
FluentValidation.DependencyInjectionExtensions
dotnet add IndustryApp.Application package Microsoft.Extensions.Options
dotnet add IndustryApp.Application package Scrutor

# Infrastructure
dotnet add IndustryApp.Infrastructure package
Npgsql.EntityFrameworkCore.PostgreSQL --version 8.*
dotnet add IndustryApp.Infrastructure package EFCore.NamingConventions --
version 8.*
dotnet add IndustryApp.Infrastructure package
Microsoft.AspNetCore.Identity.EntityFrameworkCore --version 8.*
dotnet add IndustryApp.Infrastructure package System.IdentityModel.Tokens.Jwt
dotnet add IndustryApp.Infrastructure package SixLabors.ImageSharp
dotnet add IndustryApp.Infrastructure package MailKit
dotnet add IndustryApp.Infrastructure package Microsoft.Extensions.Resilience

# WebApi
dotnet add IndustryApp.WebApi package
Microsoft.AspNetCore.Authentication.JwtBearer --version 8.*
dotnet add IndustryApp.WebApi package Microsoft.EntityFrameworkCore.Design --
version 8.*
dotnet add IndustryApp.WebApi package Serilog.AspNetCore
dotnet add IndustryApp.WebApi package Swashbuckle.AspNetCore
dotnet add IndustryApp.WebApi package AspNetCore.HealthChecks.NpgSql

# Testler
dotnet add tests/IndustryApp.UnitTests package NSubstitute
dotnet add tests/IndustryApp.UnitTests package FluentAssertions
dotnet add tests/IndustryApp.IntegrationTests package
Microsoft.AspNetCore.Mvc.Testing --version 8.*
dotnet add tests/IndustryApp.IntegrationTests package
```

```
Testcontainers.PostgreSql
dotnet add tests/IndustryApp.IntegrationTests package FluentAssertions

dotnet tool install --global dotnet-ef --version 8.*
```

1.4 Yerel veritabanı

Kök dizine `docker-compose.yml` oluştur. Faz 1'de yalnızca `db` servisi vardır; `api` ve `web` Faz 4'te eklenir (T-16.1).

```
services:
  db:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: industryapp
      POSTGRES_USER: industryapp
      POSTGRES_PASSWORD: ${DB_PASSWORD:-localdev}
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U industryapp"]
      interval: 10s
volumes:
  pgdata:
```

```
docker compose up -d db
```

Yerel bağlantı dizesi user-secrets'ta tutulur, repoya girmez:

```
cd src/backend/IndustryApp.WebApi
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:Default"
"Host=localhost;Database=industryapp;Username=industryapp;Password=localdev"
dotnet user-secrets set "Jwt:SigningKey" "$(openssl rand -base64 64)"
```

1.5 Frontend iskeleti

```
cd src/frontend
npm create vite@latest industryapp-web -- --template react-ts
cd industryapp-web
npm i @mui/material @mui/icons-material @mui/x-tree-view @emotion/react
@emotion/styled
npm i react-router-dom @tanstack/react-query axios react-hook-form zod
@hookform/resolvers @microsoft/signalr
npm i -D @types/node
```

`vite.config.ts` içine proxy ekle; böylece geliştirmede CORS ayarı gerekmez:

```
server: {
  port: 5173,
  proxy: {
    '/api':  { target: 'http://localhost:5080', changeOrigin: true },
    '/hubs': { target: 'http://localhost:5080', changeOrigin: true, ws: true
},
  },
},
```

`src/` altında T-14.2'deki klasörleri boş olarak oluştur (`app/` , `shared/` , `modules/<11 modül>/{model,controller,views,components}`). `.env.example` içine `VITE_API_BASE_URL=/api` ve `VITE_HUB_URL=/hubs/notifications` yaz.

1.6 Domain: ortak taban sınıflar

Dosya: `IndustryApp.Domain/Common/BaseEntity.cs` . Olay listesi burada tutulur; `UnitOfWork` kayıttan sonra bu listeyi okur.

```
public abstract class BaseEntity<TId> : IHasDomainEvents
{
    private readonly List<IDomainEvent> _domainEvents = new();
    public TId Id { get; protected set; } = default!;
    public IReadOnlyCollection<IDomainEvent> DomainEvents => _domainEvents;
    protected void Raise(IDomainEvent domainEvent) =>
_domainEvents.Add(domainEvent);
    public void ClearDomainEvents() => _domainEvents.Clear();
}

public abstract class AuditableEntity<TId> : BaseEntity<TId>
{
    public DateTimeOffset CreatedAt { get; protected set; }
    public DateTimeOffset? UpdatedAt { get; protected set; }
    protected void Touch(DateTimeOffset now) => UpdatedAt = now;
```

```
}

public interface IDomainEvent { DateTimeOffset OccurredAt { get; } }
public interface IHasDomainEvents { IReadOnlyCollection<IDomainEvent>
DomainEvents { get; } void ClearDomainEvents(); }

public sealed class DomainException(string code, string message) :
Exception(message)
{
    public string Code { get; } = code;
}
```

Kural: Domain'de `DateTime.UtcNow` çağrılmaz; zaman her metoda `DateTimeOffset now` parametresiyle gelir. Böylece testlerde süre (ör. 10 dakikalık geri alma penceresi) kontrol edilebilir.

1.7 Domain: entity'ler ve desen sınıfları

T-5.1'deki 15 entity'yi, T-3'teki klasörlere yaz. Her entity'nin constructor'ı `private` (EF için) ve oluşturma bir `static Create(...)` fabrika metoduyla yapılır; böylece geçersiz nesne oluşamaz. Desen içeren sınıfların iskeletleri:

State (PAT-04.1) – `Domain/Reports/States/` :

```
public interface IReportState
{
    ReportStatus Status { get; }
    bool CanReceiveOffers { get; }
    void SelectOffer(AccidentReport report, Offer offer, DateTimeOffset now);
    void UndoSelection(AccidentReport report, DateTimeOffset now, TimeSpan
window);
    void Complete(AccidentReport report, DateTimeOffset now);
    void Cancel(AccidentReport report, DateTimeOffset now);
    void Expire(AccidentReport report, DateTimeOffset now);
}

// Taban sınıf: tüm işlemler varsayılan olarak reddedilir, durumlar yalnızca
izin verdiklerini ezer
public abstract class ReportStateBase : IReportState
{
    public abstract ReportStatus Status { get; }
    public virtual bool CanReceiveOffers => false;
    public virtual void SelectOffer(AccidentReport r, Offer o, DateTimeOffset
now) => Deny(nameof(SelectOffer));
    public virtual void UndoSelection(AccidentReport r, DateTimeOffset now,
TimeSpan w) => Deny(nameof(UndoSelection));
```

```
    public virtual void Complete(AccidentReport r, DateTimeOffset now) =>
Deny(nameof(Complete));
    public virtual void Cancel(AccidentReport r, DateTimeOffset now) =>
Deny(nameof(Cancel));
    public virtual void Expire(AccidentReport r, DateTimeOffset now) =>
Deny(nameof(Expire));
    protected void Deny(string op) =>
        throw new DomainException("REPORT_INVALID_STATE", $"'{Status}'
durumunda '{op}' yapılamaz.");
}

public sealed class OpenState : ReportStateBase
{
    public override ReportStatus Status => ReportStatus.Open;
    public override bool CanReceiveOffers => true;
    public override void SelectOffer(AccidentReport r, Offer o,
DateTimeOffset now) =>
        r.ApplyTransition(ReportStatus.OfferSelected, now, selectedOfferId:
o.Id);
    public override void Cancel(AccidentReport r, DateTimeOffset now) =>
r.ApplyTransition(ReportStatus.Cancelled, now);
    public override void Expire(AccidentReport r, DateTimeOffset now) =>
r.ApplyTransition(ReportStatus.Expired, now);
}
// OfferSelectedState: UndoSelection (süre kontrolü), Complete, Cancel
// CompletedState, CancelledState, ExpiredState: hiçbir işlemi ezmez (son
durumlar)
```

`ReportStateFactory.For(ReportStatus status)` her duruma tek bir örnek döner (durum sınıfları durumsuz olduğu için paylaşılabilir). `AccidentReport` durumu `Status` alanında saklar, `private IReportState State => ReportStateFactory.For(Status);` ile state nesnesine erişir ve her işlemi ona devreder. `ApplyTransition` metodu `internal`'dır; yalnızca state sınıfları çağırabilir, durumu değiştirir ve uygun domain olayını (`OfferSelectedEvent`, `ReportExpiredEvent` vb.) üretir.

Builder (PAT-02.1) – `Domain/Reports/AccidentReportBuilder.cs`: `WithCustomer`, `WithVehicleType`, `AddDamage(categoryId, severity, note)`, `AddPhoto(photoId, storageKey, meta)`, `AtLocation(city, district, lat, lon, address)`, `WithDescription`, `Build(DateTimeOffset now, TimeSpan ttl)`. `Build` en az 1 hasar, 1 fotoğraf, konum ve araç türü yoksa `DomainException("REPORT_INCOMPLETE")` fırlatır; geçerliyse `AccidentReportCreatedEvent` üreten raporu döner.

Composite (PAT-02.2) – `Domain/Catalog/`: `IDamageComponent { int Id; string Name; IReadOnlyList<IDamageComponent> Children; bool IsLeaf; bool Contains(int categoryId); }`. `DamageCategory` bu arayüzü uygular; `Contains` kendisini ve alt ağacı

özyinelemeli arar. Frontend'deki ağaç ve admin kategori yönetimi aynı arayüzle çalışır.

Decorator – fiyat (PAT-03.4) – `Domain/Offers/Pricing/`:

```
public interface IPriceComponent { decimal Total { get; }
IReadOnlyList<PriceLine> Lines { get; } }
public sealed record PriceLine(PriceItemType Type, string? Description,
decimal Amount);

public sealed class BasePrice(decimal laborAmount, string? description) :
IPriceComponent
{
    public decimal Total => laborAmount;
    public IReadOnlyList<PriceLine> Lines => [new(PriceItemType.Labor,
description, laborAmount)];
}

public abstract class PriceComponentDecorator(IPriceComponent inner,
PriceItemType type,
                                              decimal amount, string?
description) : IPriceComponent
{
    public decimal Total => inner.Total + amount;
    public IReadOnlyList<PriceLine> Lines => [.. inner.Lines, new(type,
description, amount)];
}
public sealed class PartsCost(IPriceComponent i, decimal a, string? d) :
PriceComponentDecorator(i, PriceItemType.Parts, a, d);
public sealed class PaintCost(IPriceComponent i, decimal a, string? d) :
PriceComponentDecorator(i, PriceItemType.Paint, a, d);
public sealed class WarrantyCost(IPriceComponent i, decimal a, string? d) :
PriceComponentDecorator(i, PriceItemType.Warranty, a, d);
public sealed class OtherCost(IPriceComponent i, decimal a, string? d) :
PriceComponentDecorator(i, PriceItemType.Other, a, d);
```

`Offer.Create(...)` bir `IPriceComponent` alır; `TotalPrice = component.Total` ve `OfferPriceItem` satırları `component.Lines`'tan üretilir.

Proxy (PAT-04.3) – `Domain/Contacts/`:

```
public sealed class ContactInfoProxy(IContactInfo real, AccidentReport
report, Guid viewerId) : IContactInfo
{
    public bool IsRevealed => report.IsMatchedWith(viewerId);
    public string FullName => real.FullName;
    public string Phone => IsRevealed ? real.Phone : Mask(real.Phone);
    public string Email => IsRevealed ? real.Email : "-";
    private static string Mask(string? phone) =>
        string.IsNullOrWhiteSpace(phone) || phone.Length < 2 ? "-" : $"05**
*** ** {phone[^2..]}";
}
```

Kalan entity'ler: `CustomerProfile`, `MechanicProfile` (`Approve`, `Reject`, `SetAvailability`), `MechanicSpecialty`, `VehicleType`, `LegalDocument`, `UserConsent`, `ReportDamageItem`, `ReportPhoto`, `ReportMatch`, `ReportStatusHistory`, `Offer` (`Accept`, `MarkNotSelected`, `Reopen`, `Withdraw`, `Close`, `UpdatePricing`), `OfferPriceItem`, `Notification`. Olaylar `Domain/Events/` altında 12 `sealed record` olarak yazılır (T-9.2'deki alanlarla).

1.8 Domain birim testleri

`tests/IndustryApp.UnitTests/Domain/` altına şu testleri yaz: `OpenState_SelectOffer_MovesToOfferSelected` (TC-04.2.1), `CompletedState_SelectOffer_Throws` (TC-04.2.2), `Builder_MissingPhoto_Throws` (TC-02.3.1), `PriceDecorators_TotalEqualsSumOfLines` (TC-03.3.1), `ContactProxy_NotMatched_ReturnsMaskedPhone` (TC-04.4.1). `Undo` testleri Faz 2'de komutla birlikte yazılır.

1.9 Kalıcılık: DbContext ve configuration'lar

Dosya: `Infrastructure/Identity/ApplicationUser.cs`:

```
public sealed class ApplicationUser : IdentityUser<Guid>
{
    public string FullName { get; set; } = default!;
    public bool IsActive { get; set; } = true;
    public DateTimeOffset CreatedAt { get; set; }
    public DateTimeOffset? LastLoginAt { get; set; }
}
public sealed class ApplicationRole : IdentityRole<Guid> { }
```

Dosya: `Infrastructure/Persistence/AppDbContext.cs`:

```
public sealed class AppDbContext(DbContextOptions<AppDbContext> options)
    : IdentityDbContext<ApplicationUser, ApplicationRole, Guid>(options)
{
    public DbSet<AccidentReport> AccidentReports => Set<AccidentReport>();
    public DbSet<Offer> Offers => Set<Offer>();
    // ... T-5.1'deki diğer entity'ler

    protected override void OnModelCreating(ModelBuilder b)
    {
        base.OnModelCreating(b);
        b.Entity<ApplicationUser>().ToTable("users");
        b.Entity<ApplicationRole>().ToTable("roles");
        b.Entity<IdentityUserRole<Guid>>().ToTable("user_roles");
        b.Entity<IdentityUserClaim<Guid>>().ToTable("user_claims");
        b.Entity<IdentityUserLogin<Guid>>().ToTable("user_logins");
        b.Entity<IdentityUserToken<Guid>>().ToTable("user_tokens");
        b.Entity<IdentityRoleClaim<Guid>>().ToTable("role_claims");
        b.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

Kayıt (`Infrastructure/DependencyInjection.cs`): `services.AddDbContext<AppDbContext>(o => o.UseNpgsql(conn).UseSnakeCaseNamingConvention());`

Her entity için `Persistence/Configurations/<Entity>Configuration.cs` yaz. Kolon uzunlukları, CHECK kısıtları ve indeksler T-6.3–T-6.7'deki DDL ile birebir aynı olmalıdır. Örnek:

```
public sealed class OfferConfiguration : IEntityTypeConfiguration<Offer>
{
    public void Configure(EntityTypeBuilder<Offer> b)
    {
        b.ToTable("offers", t =>
        {
            t.HasCheckConstraint("ck_offers_total_price", "total_price > 0");
            t.HasCheckConstraint("ck_offers_status",
                "status IN
('Pending','Accepted','NotSelected','Withdrawn','Closed')");
        });
        b.Property(x => x.TotalPrice).HasPrecision(12, 2);
        b.Property(x =>
x.Currency).HasMaxLength(3).IsFixedLength().HasDefaultValue("TRY");
        b.Property(x => x.Status).HasConversion<string>().HasMaxLength(20);
        b.HasIndex(x => new { x.ReportId, x.MechanicId }).IsUnique();
```

```
        b.HasIndex(x => x.ReportId).IsUnique()
            .HasFilter("status =
'Accepted'").HasDatabaseName("ux_offers_one_accepted_per_report");
        b.HasMany(x => x.PriceItems).WithOne().HasForeignKey(x =>
x.OfferId).OnDelete(DeleteBehavior.Cascade);
        b.UseXminAsConcurrencyToken();
    }
}
```

`AccidentReport` configuration'ında ayrıca: `Status` metin dönüşümü, `CHECK (status <> 'OfferSelected' OR selected_offer_id IS NOT NULL)`, `SelectedOfferId` için `offers` tablosuna opsiyonel FK, `ix_accident_reports_expiry` kısmi indeksi (`HasFilter("status = 'Open'")`), `UseXminAsConcurrencyToken()`. `ReportStatusHistory.Snapshot` için `HasColumnType("jsonb")`.

1.10 İlk migration ve DDL karşılaştırması

```
cd src/backend
dotnet ef migrations add Initial -p IndustryApp.Infrastructure -s
IndustryApp.WebApi -o Persistence/Migrations
dotnet ef migrations script -p IndustryApp.Infrastructure -s
IndustryApp.WebApi -o ../../database/initial.sql
dotnet ef database update -p IndustryApp.Infrastructure -s IndustryApp.WebApi
```

`database/initial.sql` çıktısını T-6'daki DDL ile tablo tablo karşılaştır: 21 uygulama tablosu, 4 Identity yardımcı tablosu, tüm CHECK'ler ve 10 indeks (T-6.7) oluşmuş olmalı. Fark varsa configuration düzeltilir, migration silinip yeniden üretilir (henüz kimse kullanmadığı için güvenli).

1.11 Referans veri

`database/reference-data.sql` dosyasını T-6.8'deki içerikle oluştur, diğer 1. seviye kategorilerin (Boya, Mekanik, Elektrik, Cam, Lastik ve jant) alt kırılımlarını da aynı kalıpla ekle. `database/admin-bootstrap.sql` dosyasını T-6.9'daki içerikle oluştur (Faz 4'te üretimde kullanılır).

```
docker compose exec -T db psql -U industryapp -d industryapp <
database/reference-data.sql
```

Kontrol: `SELECT level, count(*) FROM damage_categories GROUP BY level;` üç seviyeyi de döndürmeli; `roles` tablosunda 3 satır olmalı.

1.12 Application soyutlamaları ve repository'ler

1. `Application/Abstractions/` altına T-8.1'deki 11 altyapı interface'ini ve T-8.2'deki 11 repository interface'ini yaz.
2. `Application/Common/` altına `Result`, `Result<T>`, `Error(string Code, string Message, ErrorType Type)`, `PagedResult<T>`, `ErrorType` enum'u (Validation, Unauthorized, Forbidden, NotFound, Conflict, BusinessRule) ve Options sınıflarını (`ReportsOptions`, `OffersOptions`, `MatchingOptions`, `PhotosOptions`, `EmailOptions`, `ResilienceOptions`, `JwtOptions`) yaz.
3. `Infrastructure/Persistence/Repositories/` altına her repository'yi yaz. Liste metotları `AsNoTracking()` ve `Select` ile doğrudan response modeline projekte eder.
4. `UnitOfWork` olay toplama ve hata çevirme işini yapar:

```
public sealed class UnitOfWork(AppDbContext db, IEventDispatcher dispatcher)
: IUnitOfWork
{
    public async Task<int> SaveChangesAsync(CancellationToken ct)
    {
        var sources = db.ChangeTracker.Entries<IHasDomainEvents>().Select(e
=> e.Entity)
                        .Where(e => e.DomainEvents.Count > 0).ToList();
        var events = sources.SelectMany(e => e.DomainEvents).ToList();
        int affected;
        try { affected = await db.SaveChangesAsync(ct); }
        catch (DbUpdateException ex) when (ex.InnerException is
PostgresException { SqlState: "23505" })
        { throw new ConflictException("UNIQUE_VIOLATION", "Kayıt zaten mevcut
veya çakışan bir işlem yapıldı."); }
        catch (DbUpdateConcurrencyException)
        { throw new ConflictException("CONCURRENCY_CONFLICT", "Kayıt başka
bir işlem tarafından değiştirildi."); }
        sources.ForEach(s => s.ClearDomainEvents());
        if (events.Count > 0) await dispatcher.EnqueueAsync(events, ct);   //
yalnızca kayıt başarılıysa
        return affected;
    }
}
```

1.13 Olay altyapısı (PAT-03.1)

Dosya: `Infrastructure/Events/InProcessEventDispatcher.cs` ve `EventProcessingBackgroundService.cs`:

```
public sealed class InProcessEventDispatcher : IEventDispatcher
{
    private readonly Channel<IDomainEvent> _channel =
Channel.CreateUnbounded<IDomainEvent>();
    public ChannelReader<IDomainEvent> Reader => _channel.Reader;
    public async ValueTask EnqueueAsync(IReadOnlyCollection<IDomainEvent>
events, CancellationToken ct)
    { foreach (var e in events) await _channel.Writer.WriteAsync(e, ct); }
}

public sealed class EventProcessingBackgroundService(InProcessEventDispatcher
dispatcher,
    IServiceScopeFactory scopes, ILogger<EventProcessingBackgroundService>
log) : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        await foreach (var evt in dispatcher.Reader.ReadAllAsync(ct))
        {
            using var scope = scopes.CreateScope();
            var handlerType =
typeof(IEventHandler<>).MakeGenericType(evt.GetType());
            foreach (var handler in
scope.ServiceProvider.GetServices(handlerType))
            {
                try { await ((dynamic)handler!).HandleAsync((dynamic)evt,
ct); }
                catch (Exception ex) { log.LogError(ex, "{Event} işlenemedi:
{Handler}", evt.GetType().Name, handler!.GetType().Name); }
            }
        }
    }
}
```

Geçici çözüm uyarısı: Kuyruk bellekte olduğu için süreç kapanırsa işlenmemiş olaylar kaybolur (T-9.1). Tek sunuculu ilk sürüm için kabul edildi; kalıcı çözüm Outbox'tır.

Performans notu: `dynamic` çağrısı her olayda küçük bir yansıma maliyeti getirir. Olay sayısı saniyede birkaç yüzün altında kaldığı sürece önemsizdir; gerekirse handler başına derlenmiş delegate önbelleğiyle değiştirilir.

1.14 API iskeleti

`Program.cs` Faz 1 sonunda şunları içerir: Serilog (`UseSerilog` , console + `logs/api-.log` günlük dosya), `AddApplication()` ve `AddInfrastructure(config)` çağrıları, controller'lar + camelCase JSON, Swagger, `AddHealthChecks().AddNpgSql(conn)` , `AddExceptionHandler<GlobalExceptionHandler>()` + `AddProblemDetails()` , `CorrelationIdMiddleware` (her yanıta `X-Correlation-Id` başlığı ve log bağlamı), `MapHealthChecks("/health")` .

`WebApi/Controllers/ApiControllerBase.cs` :

```
[ApiController]
public abstract class ApiControllerBase : ControllerBase
{
    protected IActionResult ToActionResult(Result result) =>
        result.IsSuccess ? NoContent() : Problem(result.Error!);
    protected IActionResult ToActionResult<T>(Result<T> result, Func<T,
IActionResult>? onSuccess = null) =>
        result.IsSuccess ? (onSuccess?.Invoke(result.Value!) ??
Ok(result.Value)) : Problem(result.Error!);

    private ObjectResult Problem(Error e)
    {
        var status = e.Type switch
        {
            ErrorType.Validation => 400, ErrorType.Unauthorized => 401,
ErrorType.Forbidden => 403,
            ErrorType.NotFound => 404, ErrorType.Conflict => 409, _ => 422
        };
        var pd = new ProblemDetails { Status = status, Title = e.Message };
        pd.Extensions["code"] = e.Code;
        pd.Extensions["traceId"] = HttpContext.TraceIdentifier;
        return new ObjectResult(pd) { StatusCode = status };
    }
}
```

`GlobalExceptionHandler` (`IExceptionHandler`): `DomainException` → 409 + kendi kodu, `ConflictException` → 409, `ValidationException` → 400 + alan hataları, diğerleri → 500 (iç ayrıntı gösterilmeden, `traceId` ile loglanır). Yerel çalıştırma portu `launchSettings.json` içinde `http://localhost:5080` olarak ayarlanır.

1.15 PA1 teslimi (Faz 1 ile paralel)

PA1 kod istemez; bu faz sürerken teknik tasarım dokümanından kesilerek hazırlanır ve `docs/PA1/` altına konur. Her dokümanın başında içindekiler, yazarlar ve dokümana özel görev matrisi bulunur.

1. **Project Definition Document:** proje adı, özet, hedefler, kapsam (dahil/hariç), hedef kitle, en az 5 temel özellik, teslimatlar, en az 3 başarı ölçütü. Kaynak: T-1, T-0.2.
2. **Project Plan Document:** kapsam, roller, hedefler, bu rehberdeki 4 fazın tarihli Gantt'ı ve PA teslim tarihleriyle eşlemesi, kaynak planı (T-2), risk yönetimi (bellek içi kuyruk, e-posta sağlayıcısı, ekip içi iş yükü, dağıtım), iletişim planı, değişiklik yönetimi (her değişiklik önce teknik tasarım dokümanında güncellenir, sonra koda geçer), efor dağılımı, kabul testleri ve kriterleri (T-19.2).
3. **Software Requirements Specification:** ürün perspektifi, fonksiyonlar, kullanıcı özellikleri, kısıtlar, UML use case diyagramı, UC-01..UC-04 (ana ve alternatif akışlarıyla, T-7), fonksiyonel olmayan gereksinimler (kullanılabilirlik, performans p95 < 500 ms, taşınabilirlik: tarayıcı + Docker, güvenilirlik), dış arayüzler (SMTP, SignalR).
4. **Mimari genel bakış:** T-4'teki diyagram ve açıklama.
5. **Desen–use case eşleme matrisi:** T-10 tablosu ve gerekçeler (her use case için en az bir desen koşulunu fazlasıyla karşılar).
6. Dokümanlar GitHub'a UC numaralı commit'lerle yüklenir; her ekip üyesi kendi hesabıyla commit atar.

Faz 1 kontrol listesi

- [ ] `dotnet build` ve `dotnet test` yeşil; 5 domain testi geçiyor
- [ ] `docker compose up -d db` ve `dotnet ef database update` sonrası 25 tablo oluşmuş
- [ ] `reference-data.sql` çalıştırılmış; roller, araç türleri ve 3 seviyeli kategori ağacı var
- [ ] `dotnet run` sonrası `http://localhost:5080/swagger` ve `/health` açılıyor
- [ ] `npm run dev` ile boş React uygulaması `http://localhost:5173`'te açılıyor
- [ ] PA1 dokümanları docs/PA1 altında; `main`'e birleştirildi, `v0.1` etiketi atıldı

Faz 2 – Kimlik ve UC-04 (PA2)

Faz 2 sonunda kullanıcılar ekrandan kaydolup giriş yapabilir, müşteri teklifleri görüp seçebilir, seçimi süre içinde geri alabilir ve eşleşme ekranında karşı tarafın iletişim bilgisini görür. State, Command, Proxy, Strategy (sıralama), Factory Method ve Observer bu fazda çalışır hale gelir. PA2 bu fazın çıktısıyla teslim edilir.

Çıktı: `v0.2` etiketi + PA2 teslimi · **Kapsadığı eski adımlar:** F3, F4 (+ F7'deki Factory Method ve sözleşme onayı öne alındı)

Neden Factory Method ve sözleşme onayı bu fazda: Kayıt akışını önce `if/else` ile yazıp Faz 4'te fabrikaya çevirmek aynı kodu iki kez yazmak demektir. Üretimde sözleşme onayı olmadan hesap açılması da KVKK açısından kabul edilemez. İkisi de kayıtla birlikte ilk seferde doğru kurulur.

2.1 Kimlik altyapısı

1. **Identity kaydı** (`Infrastructure/DependencyInjection.cs`):

```
services.AddIdentityCore<ApplicationUser>(o =>
{
    o.Password.RequiredLength = 8;
    o.Password.RequireDigit = true;
    o.Password.RequireNonAlphanumeric = false;
    o.Password.RequireUppercase = false;
    o.Lockout.MaxFailedAccessAttempts = 5;
    o.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    o.User.RequireUniqueEmail = true;
})
.AddRoles<ApplicationRole>()
.AddEntityFrameworkStores<AppDbContext>()
.AddSignInManager();
```

2. **`JwtTokenService`** (`Infrastructure/Identity/`): access token'a şu claim'ler yazılır: `sub` (UserId), `email` , `name` , `role` (bir veya birden fazla), `mechanic_status` (yalnızca ustada). Süre `Jwt:AccessTokenMinutes` . Refresh token 64 baytlık rastgele değerdir; kullanıcıya düz hali, veritabanına SHA-256 özeti gider.
3. **Refresh token rotasyonu** (`AuthService.RefreshAsync`): özeti bul → süresi geçmiş veya iptal edilmişse reddet → iptal edilmiş bir token tekrar gelmişse kullanıcının tüm aktif token'larını iptal et (çalıntı token belirtisi) → eskisini `revoked_at` + `replaced_by_token_hash` ile kapat → yenisini üret.
4. **`CurrentUser`** : `IHttpContextAccessor` üzerinden `sub` claim'ini okur; arka plan işlerinde `SystemCurrentUser` kullanılır (UserId = `Guid.Empty` , `actor_user_id` NULL yazılır).
5. **`IdentityService`** : `UserManager.CreateAsync` , `AddToRoleAsync` , `CheckPasswordSignInAsync` (kilit sayacı dahil) sarmalanır; Identity hataları `Error` nesnesine çevrilir (`DuplicateEmail` → `EMAIL_ALREADY_USED`).
6. **JWT doğrulaması ve politikalar** (`WebApi/Extensions/AuthExtensions.cs`):

```
services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.TokenValidationParameters = new()
```

```
        {
            ValidIssuer = jwt.Issuer, ValidAudience = jwt.Audience,
            IssuerSigningKey = new
SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.SigningKey)),
            ValidateIssuer = true, ValidateAudience = true, ValidateLifetime
= true,
            ValidateIssuerSigningKey = true, ClockSkew =
TimeSpan.FromSeconds(30),
            NameClaimType = "name", RoleClaimType = "role"
        };
        o.Events = new JwtBearerEvents
        {
            // WebSocket başlık gönderemez: hub bağlantısında token sorgu
parametresinden okunur
            OnMessageReceived = ctx =>
            {
                var token = ctx.Request.Query["access_token"];
                if (!string.IsNullOrEmpty(token) &&
ctx.HttpContext.Request.Path.StartsWithSegments("/hubs"))
                    ctx.Token = token;
                return Task.CompletedTask;
            }
        };
    });

services.AddAuthorizationBuilder()
    .AddPolicy("CustomerOnly", p => p.RequireRole(RoleNames.Customer))
    .AddPolicy("MechanicAny", p => p.RequireRole(RoleNames.Mechanic))
    .AddPolicy("ApprovedMechanic", p =>
p.RequireRole(RoleNames.Mechanic).RequireClaim("mechanic_status",
"Approved"))
    .AddPolicy("AdminOnly", p => p.RequireRole(RoleNames.Admin));
```

2.2 Kayıt: Factory Method (PAT-01.1) ve sözleşme onayı

1. `Application/Accounts/Factories/IUserProfileFactory.cs` (imza T-8.5).
2. `CustomerProfileFactory`: `Role = "Customer"`, `RequiredLegalDocumentCodes = ["TERMS_OF_USE", "CUSTOMER_KVKK_NOTICE"]`, `CreateProfileAsync` gelen isteği `IRegisterCustomerRequest`'e çevirir ve `CustomerProfile.Create(...)` ekler.
3. `MechanicProfileFactory`: `Role = "Mechanic"`, `RequiredLegalDocumentCodes = ["TERMS_OF_USE", "MECHANIC_SERVICE_AGREEMENT"]`, vergi numarası tekilliğini kontrol eder, `MechanicProfile.Create(...)` (`PendingApproval`) ve uzmanlık satırlarını ekler. Uzmanlık id'lerinin 1. seviye aktif kategori olduğu doğrulanır.
4. `AuthService.RegisterAsync` akışı:

```
public async Task<Result<RegistrationResponse>>
RegisterAsync(IRegisterUserRequest request, CancellationToken ct)
{
    var factory = _factories.SingleOrDefault(f => f.Role == request.Role);
    if (factory is null) return Errors.Auth.UnknownRole;

    var docs = await
_legalDocs.GetCurrentByCodesAsync(factory.RequiredLegalDocumentCodes, ct);
    if (!docs.All(d => request.AcceptedLegalDocumentIds.Contains(d.Id)))
return Errors.Auth.ConsentRequired;

    var userId = await _identity.CreateUserAsync(request.ToIdentityRequest(),
ct);   // rol ataması dahil
    if (userId.IsFailure) return userId.Error!;

    var profile = await factory.CreateProfileAsync(userId.Value, request,
ct);       // Factory Method
    if (profile.IsFailure) return profile.Error!;

    await _consents.AddRangeAsync(docs.Select(d =>
UserConsent.Create(userId.Value, d.Id, _clock.UtcNow,
_currentUser.IpAddress)), ct);
    await _uow.SaveChangesAsync(ct);
    return new RegistrationResponse(userId.Value, request.Role,
RequiresApproval: request.Role == RoleNames.Mechanic);
}
```

Tutarlılık notu: Identity kullanıcıyı kendi `SaveChanges` çağrısıyla yazar. Kullanıcı ile profilin aynı transaction'da kalması için `AuthService` işlemi `IUnitOfWork.BeginTransactionAsync` ile açılan bir transaction içinde yapar; profil adımı başarısız olursa kullanıcı da geri alınır. Bu metot `IUnitOfWork` interface'ine eklenir.

5. Validator'lar: `RegisterCustomerValidator`, `RegisterMechanicValidator` (`AbstractValidator<IRegister...Request>`): e-posta biçimi, telefon `^05\d{9}$`, şifre ≥ 8 ve rakam, vergi no 10–11 hane, enlem/boylam aralığı, en az 1 uzmanlık, en az 1 onaylı sözleşme.
6. `CatalogController` içinde yalnızca `GET /api/catalog/legal-documents?role=` bu fazda yazılır (kayıt formu sözleşmeleri göstermek için). Kategori ağacı da usta kayıt formunda gerektiği için `GET /api/catalog/damage-categories` aynı adımda eklenir.

2.3 AuthController

```
[Route("api/auth")]
public sealed class AuthController(IAuthService auth) : ApiControllerBase
{
    [HttpPost("register/customer"), AllowAnonymous,
EnableRateLimiting("register")]
    public async Task<IActionResult> RegisterCustomer(RegisterCustomerRequest
request, CancellationToken ct)
        => ToActionResult(await auth.RegisterAsync(request, ct));

    [HttpPost("register/mechanic"), AllowAnonymous,
EnableRateLimiting("register")]
    public async Task<IActionResult> RegisterMechanic(RegisterMechanicRequest
request, CancellationToken ct)
        => ToActionResult(await auth.RegisterAsync(request, ct));

    [HttpPost("login"), AllowAnonymous, EnableRateLimiting("login")]
    public async Task<IActionResult> Login(LoginRequest request,
CancellationToken ct)
        => ToActionResult(await auth.LoginAsync(request, ct));

    [HttpPost("refresh"), AllowAnonymous]
    public async Task<IActionResult> Refresh(RefreshTokenRequest request,
CancellationToken ct)
        => ToActionResult(await auth.RefreshAsync(request, ct));

    [HttpPost("logout"), Authorize]
    public async Task<IActionResult> Logout(RefreshTokenRequest request,
CancellationToken ct)
        => ToActionResult(await auth.LogoutAsync(request, ct));
}
```

İki kayıt endpoint'i farklı record alır ama aynı `RegisterAsync` metodunu çağırır; hangi profilin üretileceğine Factory Method karar verir. `EnableRateLimiting` politikaları (login ve register) bu fazda RateLimiting ayarlarından okunarak Program.cs içinde kaydedilir; tanımsız bir politikaya atıf yapan attribute uygulamanın açılmasını engeller. Faz 4'te upload politikası eklenir ve tüm limitler doğrulanır.

2.4 Frontend ortak katmanı

1. **`shared/http/axiosClient.ts`**: `baseURL = import.meta.env.VITE_API_BASE_URL`; istek interceptor'ı access token'ı ekler; yanıt interceptor'ı 401'de tek bir refresh isteği atar ve bekleyen istekleri sıraya alır:

```
let refreshing: Promise<string | null> | null = null;

client.interceptors.response.use(undefined, async (error) => {
  const original = error.config;
  if (error.response?.status !== 401 || original._retried) throw
toApiError(error);
  original._retried = true;
  refreshing ??= refreshAccessToken().finally(() => { refreshing = null; });
  const token = await refreshing;
  if (!token) { authEvents.emit('logout'); throw toApiError(error); }
  original.headers.Authorization = `Bearer ${token}`;
  return client(original);
});
```

`toApiError` ProblemDetails'teki `code` alanını okur; `shared/errors/errorMessages.ts` bu kodu Türkçe mesaja çevirir, karşılığı yoksa genel bir mesaj gösterir.

2. **`shared/auth/`**: `tokenStorage` (access bellekte, refresh `localStorage`'da; T-14.6 risk notu geçerli), `AuthProvider` (giriş, çıkış, `me`), `useAuth()`, `RequireRole` guard (rol uymuyorsa `/403`).
3. **`app/layouts/`**: `PublicLayout`, `CustomerLayout`, `MechanicLayout`, `AdminLayout`. Menüler bu fazda layout içinde sabit tanımlanır; Faz 4'te backend'deki Abstract Factory'den gelir.
4. **`shared/format/`**:

```
export const formatDate = (value?: string | null, withTime = false): string
=> {
  if (!value) return '-';
  const d = new Date(value);
  if (Number.isNaN(d.getTime())) return '-';
  const p = (n: number) => String(n).padStart(2, '0');
  const date = `${p(d.getDate())}/${p(d.getMonth() + 1)}/${d.getFullYear()}`;
  return withTime ? `${date} ${p(d.getHours())}:${p(d.getMinutes())}` : date;
};

export const labelOf = <T extends string>(map: Record<T, string>, value?: T |
string | null): string =>
  value && value in map ? map[value as T] : '-';

export const formatMoney = (amount?: number | null, currency = 'TRY'): string
=>
  amount == null ? '-' : new Intl.NumberFormat('tr-TR', { style: 'currency',
currency }).format(amount);
```

5. **`app/router.tsx`**: T-14.3'teki route'lar tanımlanır; bu fazda 1–8, 12 ve rol yönlendirmesi aktif, diğerleri "yakında" sayfasıyla yer tutar.

2.5 Kimlik ekranları

- `LoginView`: e-posta, şifre; başarıda role göre yönlendirme (T-14.3).
- `RegisterRoleSelectView`: iki kart (Müşteri / Usta).
- `RegisterCustomerView`: ortak alanlar + il/ilçe + sözleşme onay kutuları (metinler dialog'da açılır).
- `RegisterMechanicView`: ortak alanlar + atölye, vergi no, adres, konum ("Konumumu kullan" butonu `navigator.geolocation`), uzmanlık çoklu seçimi (kategori ağacının 1. seviyesi) + sözleşmeler.
- `ErrorView`: 403 ve 404.
- Formlar React Hook Form + Zod; Zod şemaları backend validator kurallarıyla aynı tutulur.

Kontrol: Ekrandan 1 müşteri ve 3 usta kaydol. Ustaları yerelde onayla:

```
UPDATE mechanic_profiles SET approval_status = 'Approved', approved_at =
now()
WHERE user_id IN (SELECT id FROM users WHERE normalized_email LIKE
'USTA%@TEST.LOCAL');
```

Ustalar çıkış yapıp tekrar girince token'larında `mechanic_status = Approved` olur.

2.6 UC-04 için yerel test verisi

UC-02 ve UC-03 henüz olmadığından rapor ve teklifler yalnızca yerelde, `database/dev/uc04-dev-data.sql` ile eklenir. Kullanıcılar ekrandan oluşturulduğu için betik onları e-postayla bulur. **Bu betik üretimde çalıştırılmaz ve Faz 3 sonunda silinir.**

```
DO $$
DECLARE
    c  uuid := (SELECT id FROM users WHERE normalized_email =
'MUSTERI@TEST.LOCAL');
    vt smallint := (SELECT id FROM vehicle_types WHERE code = 'CAR');
    cat integer := (SELECT id FROM damage_categories WHERE code =
'BODY_FRONT_BUMPER');
    r  uuid := gen_random_uuid();
BEGIN
    INSERT INTO accident_reports (id, customer_id, vehicle_type_id, city,
district, latitude, longitude, status, expires_at)
    VALUES (r, c, vt, 'Ankara', 'Çankaya', 39.92, 32.85, 'Open', now() +
interval '72 hours');
```

```
    INSERT INTO report_damage_items (report_id, damage_category_id, severity)
VALUES (r, cat, 'Medium');

    INSERT INTO report_matches (report_id, mechanic_id, strategy_code)
    SELECT r, u.id, 'CITY' FROM users u WHERE u.normalized_email LIKE
'USTA%@TEST.LOCAL';

    INSERT INTO offers (id, report_id, mechanic_id, total_price,
estimated_days, status)
    SELECT gen_random_uuid(), r, u.id, 3500 + 500 * row_number() OVER (ORDER
BY u.email), 5 - row_number() OVER (ORDER BY u.email), 'Pending'
    FROM users u WHERE u.normalized_email LIKE 'USTA%@TEST.LOCAL';

    INSERT INTO offer_price_items (offer_id, item_type, amount)
    SELECT id, 'Labor', total_price FROM offers WHERE report_id = r;

    RAISE NOTICE 'Rapor: %', r;
END $$;
```

Betik her çalıştırıldığında yeni bir rapor açar; geri alma ve iptal senaryoları tekrar tekrar denenebilir.

2.7 Teklif listeleme ve sıralama (PAT-04.4)

1. `Application/Offers/Sorting/` : `IOfferSortStrategy` , `PriceAscendingSort` (`OrderBy(TotalPrice)`), `NewestFirstSort` (`OrderByDescending(CreatedAt)`), `FastestCompletionSort` (`OrderBy(EstimatedDays).ThenBy(TotalPrice)`). DI'a hepsi `IOfferSortStrategy` olarak kaydedilir.
2. `OfferService.ListForReportAsync(IReportOffersQuery)` : rapor sahibi mi kontrol et (`NOT_REPORT_OWNER` → 403) → `SortKey` 'e uyan stratejiyi bul, yoksa `price` → `IOfferRepository.ListForReportAsync(reportId, strategy)` → yalnızca `Pending` , `Accepted` , `NotSelected` teklifler döner; her öğede usta atölye adı, toplam, tahmini gün, kalem kırılımı, durum.
3. `IReportService.GetForCustomerAsync` : rapor özeti, hasar kalemleri, fotoğraf id'leri, durum, `SelectedOfferId` , `UndoDeadline` (seçildiyse).

2.8 Teklif seçimi: Command (PAT-04.2) + State (PAT-04.1)

Dosya: `Application/Offers/Commands/SelectOfferCommand.cs` :

```
public sealed class SelectOfferCommand : IReportCommand
{
    private readonly AccidentReport _report;
    private readonly Offer _chosen;
    private readonly IReadOnlyList<Offer> _others;
    private readonly DateTimeOffset _now;
    private readonly TimeSpan _undoWindow;

    internal SelectOfferCommand(AccidentReport report, Offer chosen,
IReadOnlyList<Offer> offers,
                                DateTimeOffset now, TimeSpan undoWindow)
    {
        _report = report; _chosen = chosen; _now = now; _undoWindow =
undoWindow;
        _others = offers.Where(o => o.Id != chosen.Id && o.Status ==
OfferStatus.Pending).ToList();
    }

    public string Name => nameof(SelectOfferCommand);

    public void Execute()
    {
        _report.SelectOffer(_chosen, _now);                // State karar
verir: yalnızca Open kabul eder
        _chosen.Accept(_now);
        foreach (var o in _others) o.MarkNotSelected(_now);
    }

    public void Undo()
    {
        _report.UndoSelection(_now, _undoWindow);          // State: süre
dışıysa UNDO_WINDOW_EXPIRED
        _chosen.Reopen(_now);
        foreach (var o in _others) o.Reopen(_now);
    }

    public ReportCommandSnapshot CreateSnapshot() =>
        new(_chosen.Id, _others.Select(o => o.Id).ToArray());
}
```

`ReportCommandFactory.Restore(...)` snapshot'taki id'lerle teklifleri eşleştirip aynı komutu yeniden kurar (T-8.6).

`OfferSelectionService.SelectAsync` sırası:

1. Raporu teklifleriyle yükle (`GetWithOffersAsync`); yoksa `REPORT_NOT_FOUND`.

2. Sahiplik kontrolü: `report.CustomerId == currentUser.UserId`, değilse `NOT_REPORT_OWNER`.
3. Teklif bu rapora ait ve `Pending` değilse `OFFER_NOT_AVAILABLE` (409).
4. `var cmd = _commands.CreateSelectOffer(report, offer, actorId, now);` → `cmd.Execute();`
5. `ReportStatusHistory.Create(report.Id, Open, OfferSelected, cmd.Name, actorId, cmd.CreateSnapshot(), now)` ekle.
6. `SaveChangesAsync` → xmin veya kısmi unique index çakışması `CONFLICT` olarak döner.
7. `SelectionResponse(reportId, offerId, SelectedAt, UndoDeadline = SelectedAt + window)` dön.

`UndoAsync`: rapor + teklifler → sahiplik → en son `SelectOfferCommand` geçmiş kaydı → `Restore` → `Undo()` → `UndoSelectOffer` geçmiş kaydı → kaydet. Pencere süresi `IOptions<OffersOptions>.SelectionUndoWindowMinutes`.

`IReportService.CompleteAsync` ve `CancelAsync` da bu fazda yazılır; ikisi de doğrudan State'e devreder (`report.Complete(now)`, `report.Cancel(now)`), iptalde `Pending` teklifler `Close` edilir.

2.9 İletişim bilgisi: Proxy (PAT-04.3)

`ContactService.GetMatchContactAsync(reportId)`:

1. Raporu yükle; bakan kişi rapor sahibi müşteri veya kabul edilmiş teklifin ustası değilse ve `report_matches` kaydı da yoksa `NOT_MATCHED` (403).
2. Karşı tarafı belirle (müşteri bakıyorsa kazanan usta, usta bakıyorsa müşteri).
3. `IIdentityService.GetContactAsync` ile gerçek `ContactInfo`'yu al, `new ContactInfoProxy(real, report, viewerId)` ile sar.
4. `MatchContactResponse(FullName, Phone, Email, IsRevealed, WorkshopName?, Address?)` dön. Gerçek numaraya proxy dışında hiçbir yerden erişilmez.

2.10 Controller'lar

Bu fazda yazılan endpoint'ler (T-11.3 numaralarıyla): 1–6, 11, 13, 18, 19, 20, 27, 29, 30, 31, 32, 33–36. Örnek:

```
[Route("api/reports/{reportId:guid}/selection"), Authorize]
public sealed class OfferSelectionController(IOfferSelectionService
selection) : ApiControllerBase
{
    [HttpPost, Authorize(Policy = "CustomerOnly")]
    public async Task<IActionResult> Select(Guid reportId, SelectOfferBody
body, CancellationToken ct)
        => ToActionResult(await selection.SelectAsync(new
SelectOfferRequest(reportId, body.OfferId), ct));

    [HttpDelete, Authorize(Policy = "CustomerOnly")]
    public async Task<IActionResult> Undo(Guid reportId, CancellationToken
ct)
        => ToActionResult(await selection.UndoAsync(new
UndoSelectionRequest(reportId), ct));

    [HttpGet]
    public async Task<IActionResult> Get(Guid reportId, CancellationToken ct)
        => ToActionResult(await selection.GetAsync(reportId, ct));
}
```

2.11 Gerçek zamanlı bildirim altyapısı

1. `WebApi/Realtime/NotificationHub.cs`: boş `[Authorize] public sealed class NotificationHub : Hub { }`. İstemci hub'a işlem göndermez.
2. `SignalRRealtimePublisher : IRealtimePublisher` → `hub.Clients.User(userId.ToString()).SendAsync(eventName, payload, ct)`. `IUserIdProvider` `sub` claim'ini döndürecek şekilde kaydedilir.
3. `Program.cs`: `services.AddSignalR().AddJsonProtocol(o => o.PayloadSerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase);` ve `app.MapHub<NotificationHub>("/hubs/notifications");`
4. `NotificationService.NotifyAsync` (T-9.3): `notifications` kaydı → `notification.created` yayını → `notification_deliveries` içine `InApp/Sent`. E-posta kanalı Faz 3'te eklenir; bu fazda `Channels` listesinde Email gelirse `Skipped` yazılır.
5. Handler'lar: `OfferSelectedNotificationHandler`, `OfferSelectionUndoneNotificationHandler`, `ReportCancelledNotificationHandler`, `ReportCompletedNotificationHandler`, `ReportStatusRealtimeHandler` (`report.statusChanged`).

6. `NotificationsController`: liste, okunmamış sayısı, okundu, tümü okundu (endpoint 33–36).

2.12 UC-04 ekranları

1. **`shared/realtime/`**: `RealtimeProvider` giriş yapılınca bağlantıyı kurar (`accessTokenFactory`, `withAutomaticReconnect()`), yeniden bağlanınca `queryClient.invalidateQueries()` çağırır. Hook:

```
export function useRealtimeEvent<T>(eventName: string, handler: (payload: T)
=> void) {
  const connection = useRealtimeConnection();
  const ref = useRef(handler);
  ref.current = handler;
  useEffect(() => {
    if (!connection) return;
    const listener = (p: T) => ref.current(p);
    connection.on(eventName, listener);
    return () => connection.off(eventName, listener);
  }, [connection, eventName]);
}
```

2. **Model** (`modules/offers/model/`, `modules/selection/model/`, `modules/contacts/model/`): tipler ve `offersApi.listForReport`, `selectionApi.select/undo/get`, `contactsApi.get`; durum etiket haritaları (`Pending: 'Değerlendirmede'`, `Accepted: 'Seçildi'`, `NotSelected: 'Seçilmedi'` …).
3. **Controller** hook'ları: `useCustomerReport(reportId)`, `useReportOffers(reportId, sortKey)` (`offer.created/updated/withdrawn` ile cache günceller), `useSelectOffer()` ve `useUndoSelection()` (başarıda rapor ve teklif sorgularını geçersiz kılar), `useMatchContact(reportId)`, `useNotifications()` (`notification.created` ile sayaç artar).
4. **View**'lar: `CustomerReportDetailView` (rapor özeti + `OfferList` + `SortSelect` + `PriceBreakdown` açılır paneli + `ConfirmDialog`), `SelectionBanner` (UndoDeadline'a geri sayım, "Vazgeç" ve "Eşleşmeye git" butonları; süre bitince "Vazgeç" kaybolur), `MatchView` (karşı tarafın adı, `tel:` bağlantılı telefon, e-posta; `IsRevealed=false` ise maskeli), `NotificationBell` (layout'ta, son 10 bildirim, tıklayınca `payload.reportId` ile yönlenir). "Tamamlandı" ve "İptal et" butonları rapor durumuna göre görünür.

2.13 Testler

- Birim: `Undo_WithinWindow_RestoresPendingOffers` (TC-04.3.1), `Undo_AfterWindow_Throws` (TC-04.3.2), `SelectOffer_NotOwner_ReturnsForbidden`, sıralama stratejilerinin her biri için bir test.
- Entegrasyon (Testcontainers): `ConcurrentSelect_SecondRequestGets409` (TC-04.2.3; iki istek `Task.WhenAll` ile aynı anda), `Contact_NotMatchedMechanic_Gets403OrMasked` (E2E-07'nin otomatik hali). Test veritabanı her test sınıfında migration + `reference-data.sql` ile kurulur.

2.14 PA2 teslimi

1. Tüm desen sınıflarında yorum şablonu dolu (PAT-04.1–04.4, PAT-01.1, PAT-03.1, PAT-X.1, PAT-X.2).
2. `docs/PA2/SoftwareDesignDocument.pdf`: PA2'deki başlıkların tamamı, UC-04 için. Kaynak: T-4, T-5, T-6 (UC-04 tabloları), T-8, T-9, T-10, T-11, T-12, T-13, T-14, T-16. Eklenecek diyagramlar: sınıf diyagramı (State + Command + Proxy), seçim ve geri alma sekans diyagramları, bileşen diyagramı, ER diyagramı (raporlar ve teklifler), `CustomerReportDetailView` ve `MatchView` ekran görüntüleri.
3. `README.md`: kurulum (docker, user-secrets, migration, reference-data), UC-04'ün nasıl denendiği (yerel test verisi betiği), mimari özeti.
4. `TASK_MATRIX.md` ve dokümanın ilk sayfasında yazarlar ve doküman görev matrisi (ödevin zorunlu kıldığı format).

Faz 2 kontrol listesi

- [ ] Ekrandan müşteri ve usta kaydı; sözleşme onayı olmadan kayıt reddediliyor
- [ ] Onaysız usta yalnızca bekleme ekranını görüyor
- [ ] Müşteri teklifleri 3 farklı sıralamayla görüyor
- [ ] Seçim sonrası kazanan ve diğer ustalara anlık bildirim düşüyor (E2E-04)
- [ ] Geri alma süre içinde çalışıyor, süre dışında 409 dönüyor (E2E-05)
- [ ] Eşleşmeyen kullanıcı gerçek numarayı göremiyor (E2E-07)
- [ ] Aynı anda iki seçim isteğinden biri 409 alıyor
- [ ] PA2 dokümanı, README ve TASK_MATRIX repoda; `v0.2` etiketi atıldı

Faz 3 – UC-02 ve UC-03

Faz 3 sonunda sistem uçtan uca gerçek veriyle çalışır: müşteri rapor açar, eşleşen ustalara anlık ve e-posta bildirimi gider, ustalar teklif verir, müşteri Faz 2'deki ekranla seçer. Builder, Composite, iki Decorator, iki Adapter, Strategy, Chain of Responsibility ve Circuit Breaker

bu fazda devreye girer. Faz sonunda yerel test betiği silinir.

Çıktı: `v0.3` etiketi · **Kapsadığı eski adımlar:** F5, F6

3.1 Katalog

1. `ICatalogService`'in kalan metotları: `GetDamageTreeAsync` (aktif kategoriler, `ParentId`'ye göre ağaç kurularak `DamageCategoryNodeResponse { id, code, name, level, children[] }` döner) ve `GetVehicleTypesAsync`.
2. `CatalogController`: endpoint 11 ve 12 (13 Faz 2'de yazıldı).
3. **Önbellek:** Katalog nadiren değiştiği için `IMemoryCache` ile 10 dakika önbelleklenir; admin kategori değiştirdiğinde önbellek anahtarı silinir (Faz 4). Bu, rapor sihirbazının her açılışında ağaç sorgusunu tekrar çalıştırmayı önler.
4. Frontend `modules/catalog/controller/`: `useDamageTree()`, `useVehicleTypes()` (`staleTime: 10 dk`).

3.2 Dosya depolama: Adapter (PAT-02.4)

Dosya: `Infrastructure/Storage/LocalFileStorage.cs`:

```
public sealed class LocalFileStorage(IOptions<StorageOptions> options) :
IFileStorage
{
    private readonly string _root = Path.GetFullPath(options.Value.RootPath);

    private string Resolve(string key)
    {
        var full = Path.GetFullPath(Path.Combine(_root, key));
        if (!full.StartsWith(_root, StringComparison.Ordinal))          //
path traversal koruması
            throw new InvalidOperationException("Geçersiz dosya anahtarı.");
        return full;
    }

    public async Task SaveAsync(Stream content, string key, string
contentType, CancellationToken ct)
    {
        var path = Resolve(key);
        Directory.CreateDirectory(Path.GetDirectoryName(path)!);
        await using var file = File.Create(path);
        await content.CopyToAsync(file, ct);
    }

    public Task<Stream?> OpenReadAsync(string key, CancellationToken ct) =>
        Task.FromResult<Stream?>(File.Exists(Resolve(key)) ?
```

```
File.OpenRead(Resolve(key)) : null);

    public Task MoveAsync(string from, string to, CancellationToken ct)
    {
        var target = Resolve(to);
        Directory.CreateDirectory(Path.GetDirectoryName(target)!);
        File.Move(Resolve(from), target, overwrite: false);
        return Task.CompletedTask;
    }

    public Task DeleteAsync(string key, CancellationToken ct) {
File.Delete(Resolve(key)); return Task.CompletedTask; }
}
```

Yerel geliştirmede `Storage:RootPath` = `./.uploads` (gitignore'a eklenir), üretimde `/data/uploads` volume'ü.

3.3 Görsel işleme: Decorator (PAT-02.3)

Dosya: `Infrastructure/Imaging/`. Taban bileşen görseli yükler; her dekoratör bir adım ekler; en dıştaki JPEG olarak yazar.

```
public sealed class ImageSharpProcessor : IImageProcessor           // taban
bileşen: yükle
{
    public async Task<IProcessedImage> ProcessAsync(IImageInput input,
CancellationToken ct) =>
        new ProcessedImage(await Image.LoadAsync(input.Content, ct));
}

public abstract class ImageProcessorDecorator(IImageProcessor inner) :
IImageProcessor
{
    public async Task<IProcessedImage> ProcessAsync(IImageInput input,
CancellationToken ct)
    {
        var result = await inner.ProcessAsync(input, ct);
        Apply(result.Image);
        return result;
    }
    protected abstract void Apply(Image image);
}

public sealed class ResizeProcessor(IImageProcessor inner,
IOptions<PhotosOptions> o) : ImageProcessorDecorator(inner)
{
```

```
    protected override void Apply(Image img)
    {
        if (img.Width > o.Value.MaxWidthPx)
            img.Mutate(x => x.Resize(new ResizeOptions { Mode =
ResizeMode.Max, Size = new(o.Value.MaxWidthPx, 0) }));
    }
}

public sealed class ExifStripProcessor(IImageProcessor inner) :
ImageProcessorDecorator(inner)
{
    protected override void Apply(Image img)
    {
        img.Metadata.ExifProfile = null;   // GPS, cihaz ve tarih bilgisi
silinir
        img.Metadata.XmpProfile = null;
        img.Metadata.IptcProfile = null;
    }
}
// CompressionProcessor: Apply içinde kalite ayarını (Photos:JpegQuality)
sonuca yazar; kodlama PhotoService'te yapılır
```

DI kaydı T-13.6'daki gibi iç içe sarılır. HEIC desteği için ImageSharp tek başına yetmez; HEIC gelirse `UNSUPPORTED_IMAGE` döner ve frontend kullanıcıdan JPEG/PNG ister (tarayıcıların çoğu yüklemede otomatik JPEG'e çevirir). Bu kısıt Future Work'e yazılır.

Performans notu: Görsel işleme CPU yoğundur (1920 px'lik bir fotoğrafta onlarca ms). Tek istekte en fazla 1 fotoğraf yüklendiği ve istek sınırı olduğu için API'yi tıkamaz; yoğun trafikte arka plan kuyruğuna taşınması gerekir.

3.4 Fotoğraf servisi

`PhotoService.UploadTemporaryAsync(IUploadPhotoRequest)`:

1. Boyut `Photos:MaxFileSizeMb` üzerindeyse `FILE_TOO_LARGE`.
2. İlk 12 baytı okuyup **magic number** kontrolü (JPEG `FF D8 FF`, PNG `89 50 4E 47`, WebP `RIFF....WEBP`); içerik türü başlığına güvenilmez. Uymuyorsa `UNSUPPORTED_IMAGE`.
3. `IImageProcessor` zincirinden geçir, JPEG olarak `MemoryStream`'e kodla.
4. `temp/{userId}/{Guid}.jpg` anahtarıyla `IFileStorage.SaveAsync`.
5. `TemporaryPhotoResponse(tempKey, width, height, sizeBytes)` dön. Frontend önizlemeyi yerel `URL.createObjectURL` ile gösterir, anahtarı saklar.

`OpenAsync(photoId)`: fotoğrafın raporunu bul → bakan kişi rapor sahibi, eşleşen usta (`report_matches`) veya admin değilse `FORBIDDEN` → akışı `IPhotoStream(stream, contentType)` olarak dön. `PhotosController.Get` bunu `File(stream, contentType)` ile

gönderir ve `Cache-Control: private, max-age=3600` ekler.

3.5 Rapor oluşturma: Builder (PAT-02.1)

`ReportService.CreateAsync(ICreateReportRequest)` sırası:

1. Validator: araç türü var mı, 1–`MaxCountPerReport` fotoğraf anahtarı, en az 1 hasar, enlem/boylam aralığı, açıklama ≤ 1000.
2. Açık rapor sayısı `Reports:MaxOpenPerCustomer` sınırındaysa `OPEN_REPORT_LIMIT_REACHED` (422).
3. Her anahtar `temp/{currentUserId}/` ile başlamıyorsa `INVALID_PHOTO_KEY` (başkasının dosyası bağlanamaz).
4. Hasar kategori id'leri `IDamageCategoryRepository.GetByIdsAsync` ile doğrulanır; pasif veya olmayan kategori reddedilir.
5. Builder ile kur:

```
var builder = new AccidentReportBuilder()
    .WithCustomer(_currentUser.UserId)
    .WithVehicleType(request.VehicleTypeId)
    .WithDescription(request.Description)
    .AtLocation(request.City, request.District, request.Latitude,
request.Longitude, request.AddressText);
foreach (var d in request.Damages) builder.AddDamage(d.CategoryId,
d.Severity, d.Note);

var moved = new List<string>();
try
{
    foreach (var (tempKey, index) in request.TemporaryPhotoKeys.Select((k, i)
=> (k, i)))
    {
        var photoId = Guid.NewGuid();
        var finalKey = PhotoKeys.ForReport(builder.ReportId, photoId,
_clock.UtcNow);
        await _storage.MoveAsync(tempKey, finalKey, ct);
        moved.Add(finalKey);
        builder.AddPhoto(photoId, finalKey, await
_photoMeta.ReadAsync(finalKey, ct), index);
    }
    var report = builder.Build(_clock.UtcNow,
TimeSpan.FromHours(_options.ExpireAfterHours));   // eksikse hata
    await _reports.AddAsync(report, ct);
    await _uow.SaveChangesAsync(ct);                    //
AccidentReportCreatedEvent burada kuyruğa girer
```

```
    return new ReportCreatedResponse(report.Id);
}
catch
{
    foreach (var key in moved) await _storage.DeleteAsync(key, ct);   //
telafi: yarım kalan dosyaları temizle
    throw;
}
```

6. `ListMineAsync`: müşterinin raporları, en yeni önce; her satırda durum, teklif sayısı, en düşük teklif, oluşturulma tarihi.
7. `ReportsController`: endpoint 16 ve 17. `PhotosController`: endpoint 14 ve 15 (`[RequestSizeLimit]` ayardaki sınırın biraz üstüne).

3.6 Arka plan işleri

1. `ReportExpirationJob` (`BackgroundService` + `PeriodicTimer(5 dk)`): yeni scope → `ListExpiredOpenAsync(now, 100)` → her rapora `report.Expire(now)` (State karar verir) → `Pending` teklifleri `Close` → kaydet. Her rapor `ReportExpiredEvent` üretir.
2. `TemporaryUploadCleanupJob` (saatte bir): `temp/` altında 24 saatten eski dosyaları siler. `IFileStorage`'a `ListOlderThanAsync(prefix, before)` metodu eklenir.
3. Handler: `ReportExpiredNotificationHandler` → müşteriye `REPORT_EXPIRED`.
4. Test için `Reports:ExpireAfterHours` yerelde küçük bir değere çekilebilir; bunun için ayarın `double` olması yeterlidir (0.1 saat = 6 dk). E2E-08 bu yolla denenir.

3.7 Rapor sihirbazı ve müşteri ekranları

1. **Model** (`modules/reports/model/`): `CreateReportRequest`, `ReportListItem`, `CustomerReportDetail` tipleri; `reportsApi.create/listMine/get/cancel/complete`, `photosApi.uploadTemporary` (Axios `onUploadProgress`).
2. **Controller**: `useCreateReportWizard()` (React Hook Form, adım durumu, adım bazlı Zod doğrulaması), `usePhotoUpload()` (her dosya için ayrı yükleme, ilerleme, hata ve yeniden deneme), `useMyReports(page)`, `useGeolocation()`.
3. **View**: `CreateReportWizardView` (MUI `Stepper`; `PhotoUploadStep`, `DamageSelectStep` (`SimpleTreeView` + her seçili kategori için şiddet seçimi), `VehicleStep`, `LocationStep` ("Konumumu kullan" veya il/ilçe listesi), `SummaryStep`); `MyReportsView` (kart listesi, durum çipi, `formatDate`); `CustomerDashboardView` (açık rapor sayısı, son teklifler; Faz 4'te içerik Abstract Factory'den gelir).
4. Başarılı oluşturmada `/customer/reports/:id` sayfasına yönlenir; bu ekran Faz 2'den hazırdır ve teklifler geldikçe canlı dolar.

3.8 Eşleştirme: Strategy (PAT-03.2) + Chain of Responsibility (PAT-03.3)

1. **Stratejiler** (`Application/Matching/Strategies/`): `CityMatchingStrategy` (`Code = "CITY"`) → `FindCandidatesByCityAsync(city, rootCategoryIds)` ; `RadiusMatchingStrategy` (`Code = "RADIUS"`) → `FindCandidatesWithinRadiusAsync(lat, lon, DefaultRadiusKm, rootCategoryIds)` (Haversine SQL). `MatchingStrategyResolver` `Matching:Strategy` ayarındaki kodu seçer; bilinmeyen kodda açılışta hata verir (yanlış ayar sessizce geçmez).
2. **Filtre zinciri** (`Application/Matching/Filters/`), Template Method'lu taban sınıf:

```
public abstract class MechanicFilterBase : IMechanicFilter
{
    private IMechanicFilter? _next;
    public IMechanicFilter SetNext(IMechanicFilter next) { _next = next;
return next; }

    // Template Method: iskelet sabit (filtrele → boşsa dur → devret), adım
alt sınıfta
    public async Task<IReadOnlyList<MechanicCandidate>>
ApplyAsync(IMatchingContext ctx,
        IReadOnlyList<MechanicCandidate> candidates, CancellationToken ct)
    {
        var remaining = await FilterAsync(ctx, candidates, ct);
        ctx.Trace(GetType().Name, candidates.Count, remaining.Count);   //
hangi filtre kaç kişi eledi
        return remaining.Count == 0 || _next is null ? remaining : await
_next.ApplyAsync(ctx, remaining, ct);
    }
    protected abstract Task<IReadOnlyList<MechanicCandidate>>
FilterAsync(IMatchingContext ctx,
        IReadOnlyList<MechanicCandidate> candidates, CancellationToken ct);
}
```

Filtreler: `ApprovedAndActiveFilter` (onaylı + `users.is_active`), `SpecialtyFilter` (adayın uzmanlıkları raporun `RootCategoryIds` kümesiyle kesişiyor mu), `AvailabilityFilter` (`is_available`), `DailyCapacityFilter` (`CountAcceptedTodayAsync` < `daily_capacity`). `MechanicFilterChainBuilder` , `Matching:Filters` listesindeki sırayla zinciri kurar; listede olmayan filtre çalışmaz.

3. **`MatchingService.MatchAsync(reportId)`** : raporu yükle, `Open` değilse dur → strateji ile adaylar → zincir → `MaxMechanicsPerReport` kadarını al (mesafe biliniyorsa en yakınlar) → `report_matches` yaz → `MechanicsMatchedEvent` (liste boş olsa da

yayınlanır) → kaydet. Aynı rapor için ikinci kez çağrılırsa mevcut eşleşmeler atlanır (PK sayesinde idempotent).

4. **Handler'lar**: `AccidentReportCreatedMatchingHandler` → `MatchAsync`; `MechanicsMatchedNotificationHandler` → her ustaya `NEW_JOB` (InApp + Email), liste boşsa müşteriye `NO_MECHANIC_FOUND`.
5. `ctx.Trace` kayıtları `Information` seviyesinde loglanır; sunumda "bu rapor 12 adaydan 4 ustaya indi, 5'i uzmanlık, 3'ü kapasite filtresinde elendi" gösterilebilir.

3.9 Bildirim kanalları: Adapter (PAT-03.5) + Circuit Breaker (PAT-03.6)

1. `SmtpEmailChannel : INotificationChannel` (MailKit): `Email` ayarlarıyla bağlanır, `Notifications/Templates/<Type>.html` şablonunu `{{title}}`, `{{body}}`, `{{link}}` yer tutucularıyla doldurur. Gönderim hatasını exception olarak fırlatır (dayanıklılık katmanı yakalar).
2. `LoggingSmsChannel`: mesajı telefon maskeli loglar ve `Skipped` döner.
3. `ResilientNotificationChannel` (Decorator yapısında, Circuit Breaker niyetinde):

```
public sealed class ResilientNotificationChannel(INotificationChannel inner,
ResiliencePipeline pipeline,
    ILogger<ResilientNotificationChannel> log) : INotificationChannel
{
    public NotificationChannelType Channel => inner.Channel;

    public async Task<IDeliveryResult> SendAsync(INotificationMessage
message, CancellationToken ct)
    {
        try
        {
            return await pipeline.ExecuteAsync(async t => await
inner.SendAsync(message, t), ct);
        }
        catch (BrokenCircuitException)
        {
            log.LogWarning("{Channel} devresi açık, gönderim atlandı",
Channel);
            return DeliveryResult.CircuitOpen();
        }
        catch (Exception ex)
        {
            log.LogError(ex, "{Channel} gönderimi başarısız", Channel);
```

```
            return DeliveryResult.Failed(ex.Message);
        }
    }
}
```

Pipeline (`Infrastructure/Notifications/ResiliencePipelines.cs`), değerler `Resilience:Email` 'den:

```
new ResiliencePipelineBuilder()
    .AddRetry(new RetryStrategyOptions
    {
        MaxRetryAttempts = o.RetryCount, BackoffType =
DelayBackoffType.Exponential, UseJitter = true,
        Delay = TimeSpan.FromSeconds(1),
        ShouldHandle = new PredicateBuilder().Handle<SmtpCommandException>
().Handle<IOException>().Handle<TimeoutRejectedException>()
    })
    .AddCircuitBreaker(new CircuitBreakerStrategyOptions
    {
        FailureRatio = o.FailureRatio, MinimumThroughput =
o.MinimumThroughput,
        SamplingDuration = TimeSpan.FromSeconds(o.SamplingSeconds),
BreakDuration = TimeSpan.FromSeconds(o.BreakDurationSeconds)
    })
    .AddTimeout(TimeSpan.FromSeconds(o.TimeoutSeconds))
    .Build();
```

Circuit Breaker durumu kanal başına tek olmalıdır; bu yüzden pipeline ve `ResilientNotificationChannel` **singleton** kaydedilir. Scoped kaydedilseydi her istekte yeni devre oluşur ve hiç açılmazdı.

4. `NotificationService.NotifyAsync` artık `Channels` listesindeki her kanal için ilgili `INotificationChannel` 'ı çağırır ve sonucu `notification_deliveries` 'e (`Sent` , `Failed` , `CircuitOpen` , `Skipped`) yazar.
5. Yerel e-posta testi için SMTP ayarı olarak bir test SMTP sunucusu (ör. `docker run -p 1025:1025 -p 8025:8025 axllent/mailpit`) kullanılır; bu yalnızca geliştirme aracıdır, üretimde gerçek SMTP kullanılır.

3.10 Teklif verme

1. `OfferService.SubmitAsync(ISubmitOfferRequest)` :
   a. Ustanın bu raporla eşleşmesi var mı (`IsMatchedAsync`), yoksa `NOT_MATCHED` (403).
   b. `report.CanReceiveOfferFrom(mechanicId)` (State: yalnızca `Open`), değilse `REPORT_INVALID_STATE` (409).

   c. Aynı usta daha önce teklif verdiyse `OFFER_ALREADY_EXISTS` (409); frontend bunu güncelleme moduna yönlendirir.
   d. Fiyat zincirini kur: `Labor` kalemi `BasePrice`, diğerleri tiplerine göre `PartsCost`, `PaintCost`, `WarrantyCost`, `OtherCost` ile sarılır (Decorator). En az bir `Labor` kalemi zorunludur.
   e. `Offer.Create(report, mechanicId, component, estimatedDays, note, now)` → `OfferSubmittedEvent` → kaydet.
2. `UpdateAsync`: yalnızca `Pending` ve rapor `Open` iken; kalemler silinip yeniden yazılır → `OfferUpdatedEvent`.
3. `WithdrawAsync`: yalnızca `Pending` → `Withdrawn` → `OfferWithdrawnEvent`.
4. `GetMyOfferForReportAsync`, `ListMineAsync` (filtre: Pending / Accepted / NotSelected / Withdrawn).
5. `ReportService.GetForMechanicAsync(reportId)`: eşleşme kontrolü → `report_matches.viewed_at` işaretle → `MechanicJobDetailResponse` (fotoğraf id'leri, hasarlar, araç türü, ilçe düzeyinde konum, durum, **yalnızca** `MyOffer`). Müşterinin adı ve telefonu bu modelde yoktur; seçim sonrası yalnızca `MatchView` üzerinden görünür.
6. `ListMechanicJobsAsync(filter)`: `New` (eşleşmiş, teklif yok, rapor açık), `Offered`, `Won`, `Lost`.
7. Handler'lar: `OfferSubmittedNotificationHandler` (müşteriye `NEW_OFFER`), `OfferChangedRealtimeHandler` (`offer.created`, `offer.updated`, `offer.withdrawn`).
8. Controller'lar: `OffersController` (23–26, 28), `MechanicJobsController` (21, 22), `AccountController.SetAvailability` (10).

3.11 Usta onayı (YA-01)

1. `MechanicApprovalService`: `ListAsync(status)`, `ApproveAsync(id)` → `MechanicProfile.Approve(adminId, now)` → `MechanicApprovedEvent`; `RejectAsync(id, reason)` → `MechanicRejectedEvent`.
2. `MechanicApprovalNotificationHandler`: uygulama içi + e-posta.
3. `AdminMechanicsController`: endpoint 37–39.
4. **Token güncelliği:** Onaylanan ustanın elindeki access token hâlâ `mechanic_status = PendingApproval` taşır. Refresh sırasında claim'ler veritabanından yeniden okunduğu için en geç 30 dakikada güncellenir; frontend onay bildirimi (`MECHANIC_APPROVED`) geldiğinde hemen refresh çağırarak bu gecikmeyi ortadan kaldırır.

3.12 Usta ve admin ekranları

- `MechanicPendingApprovalView`: durum (bekliyor / reddedildi + gerekçe), profil bilgisi.

- `MechanicDashboardView`: yeni iş sayısı, bekleyen teklifler, kazanılan işler, müsaitlik anahtarı.
- `JobsView`: sekmeler (Yeni / Teklif verdiklerim / Kazandıklarım / Kaybettiklerim); `notification.created` tipi `NEW_JOB` gelince liste yenilenir ve pop-up (MUI `Snackbar`) çıkar.
- `JobDetailView`: fotoğraf galerisi (`/api/photos/{id}`, `Authorization` başlığıyla blob olarak çekilir), hasar listesi, `OfferForm` (dinamik kalem listesi `useFieldArray`, toplam anlık hesaplanır, tahmini gün, not); teklif varsa güncelleme modu + "Teklifi geri çek".
- `MyOffersView`: durum filtresiyle liste.
- `MechanicApprovalsView` (admin): bekleyenler tablosu, onay ve gerekçeli ret dialog'u.

3.13 Testler ve temizlik

- Birim: `FilterChain_ExcludesUnapprovedAndBusy` (TC-03.1.1), `StrategyResolver_UsesConfiguredStrategy` (TC-03.1.2), `ResilientChannel_OpensCircuitAfterFailures` (TC-03.5.1; sahte kanal hep hata fırlatır, 5. çağrıdan sonra `CircuitOpen` beklenir), `ImagePipeline_RemovesGpsExif` (TC-02.1.1; GPS etiketli örnek görsel `tests/assets/` altında), `SubmitOffer_NotMatched_Forbidden`.
- Entegrasyon: `MechanicJobDetail_NeverContainsOtherOffers` (TC-03.4.1), `CreateReport_ReturnsBeforeMatchingCompletes` (TC-02.3.2), `PendingMechanic_CannotSubmitOffer` (TC-01.3.1).
- `database/dev/uc04-dev-data.sql` silinir; artık tüm veri ekranlardan oluşur.

Faz 3 kontrol listesi

- [ ] Usta kaydolur, admin onaylar, usta e-posta ve uygulama içi bildirim alır (E2E-01)
- [ ] Müşteri 3 fotoğraflı rapor açar; eşleşen ustalarda birkaç saniye içinde pop-up çıkar (E2E-02)
- [ ] İki usta teklif verir; müşteri ekranı yenilenmeden güncellenir, ustalar birbirini görmez (E2E-03)
- [ ] SMTP ayarı bozulunca uygulama içi bildirim sürer, `CircuitOpen` kayıtları oluşur (E2E-06)
- [ ] Süresi dolan rapor `Expired` olur ve müşteri bilgilendirilir (E2E-08)
- [ ] Yüklenen fotoğraflarda EXIF/GPS yok
- [ ] Yerel test betiği silindi; `v0.3` etiketi atıldı

Faz 4 – UC-01 tamamlama, yönetim ve üretime çıkış (PA3)

Faz 4 sonunda proje gerçek alan adında, HTTPS ile, yedekli ve izlenebilir şekilde yayındadır; demo bu üretim ortamında yapılır. UC-01'in Abstract Factory kısmı, profil ve yönetim ekranları, güvenlik sıkılaştırmaları, tüm testler, test raporu ve PA3 dokümanları bu fazda tamamlanır.

Çıktı: `v1.0` etiketi + canlı sistem + PA3 teslimi · **Kapsadığı eski adımlar:** F7, F8

4.1 Rol bazlı panel: Abstract Factory (PAT-01.2)

1. `Application/Accounts/Factories/IDashboardFactory.cs` (imza T-8.5) ve üç aile:
   - `CustomerDashboardFactory`: menü (Raporlarım, Yeni rapor, Bildirimler, Profil), özet (açık rapor, bekleyen teklif, tamamlanan iş sayısı), hızlı işlem ("Hasar bildir").
   - `MechanicDashboardFactory`: menü (İşler, Tekliflerim, Bildirimler, Profil), özet (yeni iş, bekleyen teklif, bugün kabul edilen / günlük kapasite), hızlı işlem ("Müsaitliği değiştir").
   - `AdminDashboardFactory`: menü (Usta onayları, Kategoriler), özet (onay bekleyen usta, son 24 saatte açılan rapor, başarısız bildirim sayısı), hızlı işlem ("Bekleyenleri incele").
2. `AccountService.GetDashboardAsync`: kullanıcının rolüne uyan fabrikayı `IEnumerable<IDashboardFactory>` içinden seçer ve üç parçayı aynı fabrikadan üretir. Parçaların hep aynı aileden gelmesi, müşteriye usta menüsü gitmesini yapısal olarak imkânsız kılar.
3. `AccountController.Dashboard` (endpoint 7).
4. Frontend: `useDashboard()` hook'u; layout'lardaki sabit menüler kaldırılır, menü ve hızlı işlemler bu yanıttan çizilir. Menü öğesindeki `key` değeri frontend'de ikon ve route'a eşlenir (`menuRegistry.ts`); bilinmeyen `key` gösterilmez.
5. Test: `DashboardFactory_EachRole_ReturnsOwnFamily` (üç rol için menü anahtarları kesişmez).

4.2 Profil ve müsaitlik

1. `AccountService.GetMeAsync` (endpoint 6 Faz 2'de vardı; profil alanları eklenir), `UpdateCustomerProfileAsync` (il, ilçe, ad, telefon), `UpdateMechanicProfileAsync` (atölye adı, adres, konum, uzmanlıklar, günlük kapasite; vergi numarası değiştirilemez), `SetAvailabilityAsync`.
2. Telefon değişikliği `users.phone_number`'ı günceller; iletişim bilgisi tek kaynaktan okunduğu için `MatchView` otomatik yeni numarayı gösterir.
3. `AccountController`: endpoint 8, 9 (10 Faz 3'te yazıldı).

4. `ProfileView`: role göre iki form (`CustomerProfileForm`, `MechanicProfileForm`), onay durumu çipi, sözleşme onay geçmişi (hangi sürüm, hangi tarih; `formatDate`).

4.3 Kategori yönetimi ve admin paneli

1. `CategoryAdminService`: `CreateAsync` (üst kategori var ve aktif mi, seviye ≤ 3, kod benzersiz; `root_id` üstten devralınır), `UpdateAsync` (ad, sıra), `SetActiveAsync` (pasifleştirilen kategori yeni raporlarda görünmez, eski raporlar etkilenmez). Her değişiklik katalog önbelleğini temizler.
2. `AdminCatalogController`: endpoint 40, 41.
3. `CategoryManagementView`: ağaç görünümü, ekle / yeniden adlandır / pasifleştir dialog'ları.
4. `AdminDashboardView`: Abstract Factory'den gelen özet kartları; başarısız bildirim sayısı `notification_deliveries` tablosundan (Circuit Breaker'ın etkisi burada da görünür).

4.4 Bildirim sayfası ve bakım işleri

1. `NotificationsView`: sayfalı liste, okunmamışlar vurgulu, "Tümünü okundu yap", tarih `dd/MM/yyyy HH:mm`, tıklayınca `payload` ile ilgili ekrana yönlendirme.
2. `RefreshTokenCleanupJob` (günde bir): süresi dolmuş veya 30 günden eski iptal edilmiş token'ları siler.

4.5 Güvenlik sıkılaştırması

1. **İstek sınırlama** (`WebApi/Extensions/RateLimitingExtensions.cs`), değerler `RateLimiting` ayarından. Faz 2'de kaydedilen `login` ve `register` politikalarına `upload` eklenir:

```
services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
    o.AddPolicy("login", ctx => RateLimitPartition.GetFixedWindowLimiter(
        ctx.Connection.RemoteIpAddress?.ToString() ?? "unknown",
        _ => new FixedWindowRateLimiterOptions { PermitLimit =
rl.LoginPerMinute, Window = TimeSpan.FromMinutes(1) }));
    o.AddPolicy("register", ctx => RateLimitPartition.GetFixedWindowLimiter(
        ctx.Connection.RemoteIpAddress?.ToString() ?? "unknown",
        _ => new FixedWindowRateLimiterOptions { PermitLimit =
rl.RegisterPerMinute, Window = TimeSpan.FromMinutes(1) }));
    o.AddPolicy("upload", ctx => RateLimitPartition.GetFixedWindowLimiter(
        ctx.User.FindFirst("sub")?.Value ?? "anon",
        _ => new FixedWindowRateLimiterOptions { PermitLimit =
rl.UploadPerMinute, Window = TimeSpan.FromMinutes(1) }));
});
```

Caddy arkasında gerçek istemci IP'si için `app.UseForwardedHeaders()` (yalnızca Caddy konteynerinin ağına güvenecek şekilde) eklenir; aksi halde tüm kullanıcılar aynı IP'den gelmiş sayılır ve sınır herkesi birlikte keser.

2. **CORS:** `Cors:AllowedOrigins` listesi; üretimde frontend ve API aynı alan adında olduğu için liste yalnızca o alan adını içerir.
3. **HSTS ve güvenlik başlıkları:** Caddyfile'da HSTS; API'de `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`.
4. **Log maskeleme:** Serilog'a `PhoneMaskingEnricher` eklenir; `phone`, `password`, `token` adlı özellikler maskelenir veya çıkarılır.
5. **Hesap pasifleştirme:** `users.is_active = false` olan kullanıcı giriş yapamaz ve refresh alamaz; eşleştirme filtresi de bunu dikkate alır (Faz 3).
6. T-15 tablosundaki her maddenin karşılandığı tek tek işaretlenir.

4.6 Testlerin tamamlanması ve test raporu

1. T-19.1'deki 15 otomatik testin hepsi repoda ve yeşil. Eksik kalanlar bu fazda yazılır.
2. `dotnet test --collect:"XPlat Code Coverage"` ile kapsam raporu alınır; Domain ve Application için hedef %70 üzeri (desen sınıfları %90 üzeri).
3. T-19.2'deki 8 uçtan uca senaryo **üretim ortamında** elle yürütülür.
4. `docs/PA3/TestReport.xlsx`: kolonlar test no, test adı, UC/FR, adımlar, beklenen, gerçekleşen, test eden, tarih (`dd/MM/yyyy`), ortam, log/ `traceId`, durum (Pass / Fail / Blocked), not. Otomatik testler de satır olarak eklenir (ortam: CI/yerel).

4.7 Docker imajları

`src/backend/Dockerfile` (çok aşamalı; migration bundle da üretilir):

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore IndustryApp.WebApi/IndustryApp.WebApi.csproj
RUN dotnet publish IndustryApp.WebApi/IndustryApp.WebApi.csproj -c Release -o
/app/publish --no-restore
RUN dotnet tool install --global dotnet-ef --version 8.* && export
PATH="$PATH:/root/.dotnet/tools" \
 && dotnet ef migrations bundle -p IndustryApp.Infrastructure -s
IndustryApp.WebApi -o /app/publish/efbundle --self-contained -r linux-x64

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
ENV ASPNETCORE_URLS=http://+:8080
EXPOSE 8080
USER app
ENTRYPOINT ["dotnet", "IndustryApp.WebApi.dll"]
```

`src/frontend/industryapp-web/Dockerfile`:

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_BASE_URL
ARG VITE_HUB_URL
RUN npm run build

FROM caddy:2-alpine
COPY --from=build /app/dist /srv
```

Caddyfile (T-16.2) `deploy/Caddyfile` olarak tutulur ve `docker-compose.yml`'de `web` servisine `./deploy/Caddyfile:/etc/caddy/Caddyfile:ro` volume'üyle bağlanır. `docker-compose.yml` T-16.1'deki tam hale getirilir (`api` ve `web` servisleri eklenir, `db`'nin `ports` satırı üretimde kaldırılır; veritabanı dışarı açılmaz).

`deploy/.env.example`:

```
DOMAIN=industryapp.com
DB_PASSWORD=
JWT_SIGNING_KEY=
SMTP_HOST=
SMTP_USER=
SMTP_PASSWORD=
```

4.8 Sunucuya ilk dağıtım

1. Sunucu: Ubuntu LTS, 2 vCPU / 4 GB RAM. Docker ve Compose eklentisini kur; yalnızca 22, 80, 443 portlarını aç (`ufw`).
2. Alan adının A kaydını sunucu IP'sine yönlendir.
3. Repoyu klonla, `deploy/.env.example` → `.env` kopyala ve doldur (`openssl rand -base64 64` ile JWT anahtarı).
4. `docker compose up -d db`
5. Migration: `docker compose run --rm --entrypoint ./efbundle api --connection "Host=db;Database=industryapp;Username=industryapp;Password=$DB_PASSWORD"`
6. Referans veri: `docker compose exec -T db psql -U industryapp -d industryapp < database/reference-data.sql`
7. `docker compose up -d --build` → `https://<alan-adı>/health` "Healthy" dönmeli; Caddy sertifikayı otomatik alır.
8. Uygulamadan admin adayı müşteri olarak kaydolur → `admin-bootstrap.sql` e-posta düzenlenerek çalıştırılır → çıkış/giriş sonrası admin paneli açılır.
9. Sözleşme metinlerinin gerçek içeriği `legal_documents.body` alanlarına girilir (`UPDATE ... WHERE code = ... AND is_current`).
10. **Yedekleme** (sunucuda cron, her gece 03:00):

```
0 3 * * * cd /opt/IndustryApp && docker compose exec -T db pg_dump -U
industryapp -Fc industryapp > /backups/db-$(date +\%Y\%m\%d).dump \
  && tar -czf /backups/uploads-$(date +\%Y\%m\%d).tar.gz -C
/var/lib/docker/volumes/industryapp_uploads/_data . \
  && find /backups -mtime +7 -delete
```

Bir yedek, ayrı bir veritabanına `pg_restore` ile geri yüklenerek denenir; denenmemiş yedek yedek sayılmaz.

11. **Güncelleme akışı:** `git pull` → yeni migration varsa adım 5 → `docker compose up -d --build api web` .
12. T-15'teki yayın öncesi kontrol listesi tamamlanır.

4.9 PA3 teslimi ve demo

1. `docs/PA3/SoftwareDesignDocument.pdf`: PA2 dokümanı dört use case'i kapsayacak şekilde güncellenir; UC-01..UC-03 için sınıf ve sekans diyagramları (Factory Method + Abstract Factory, Builder + Decorator + Adapter, Pub-Sub + Strategy + CoR + Circuit Breaker), tam ER diyagramı, dağıtım diyagramı (T-16), değişiklik günlüğü ve Future Work (T-20).
2. `docs/PA3/TestReport.xlsx` (4.6).
3. `TASK_MATRIX.md` güncel; README'ye üretim adresi ve dağıtım adımları eklenir.
4. **Demo senaryosu (üretimde, iki tarayıcı + bir telefon):** usta kaydı ve admin onayı → müşteri rapor açar → ustada pop-up ve e-posta → iki usta teklif verir (birbirini görmez) → müşteri sıralar ve seçer → eşleşme ekranında numaralar açılır, eşleşmeyen usta maskeli görür → müşteri vazgeçer ve tekrar seçer → iş tamamlandı. Yedek senaryo: SMTP ayarı bozuk bir kopya ortamda Circuit Breaker gösterimi ve `notification_deliveries` kayıtları.
5. Demo öncesi gün üretimde tam prova yapılır. Prova kayıtları gerçek veri olarak kalır; üretim verisi silinmez, gerekirse ilgili raporlar iptal edilir.

Faz 4 kontrol listesi

- [ ] Üç rol kendi menüsünü ve özetini backend'den alıyor
- [ ] Profil düzenleme ve müsaitlik anahtarı çalışıyor
- [ ] Admin kategori ekleyip pasifleştirebiliyor; sihirbaz hemen güncelleniyor
- [ ] Rate limiting, CORS, HSTS, log maskeleme aktif; T-15 maddelerinin hepsi işaretli
- [ ] 15 otomatik test yeşil; 8 uçtan uca senaryo üretimde geçti; test raporu hazır
- [ ] Sistem HTTPS ile yayında, `/health` sağlıklı, gece yedeği alınıyor ve bir kez geri yüklendi
- [ ] PA3 dokümanları repoda; `v1.0` etiketi atıldı